

Svar på Faktafrågor och Resonemangsfrågor

Alla Kapitel (1-10)

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 1: Introduktion till maskininlärning

Faktafrågor

1. Hur hänger AI, ML och DL ihop?

AI (Artificiell Intelligens), ML (Maskininlärning) och DL (Djupinlärning) hänger ihop som en hierarki:

- **AI** är det övergripande fältet som handlar om att skapa intelligenta maskiner
- **ML** är en delmängd av AI som fokuserar på algoritmer som kan lära från data
- **DL** är en delmängd av ML som använder neurala nätverk med många lager

Förhållandet: $AI \supset ML \supset DL$

2. Vilka är de fyra problemkategorierna inom ML?

De fyra problemkategorierna inom maskininlärning är:

1. **Regression** - Prediktera kontinuerliga värden (t.ex. huspriser, lön)
2. **Klassificering** - Prediktera diskreta kategorier (t.ex. spam/ej spam, kundsegment)
3. **Dimensionsreducering** - Minska antalet variabler (t.ex. PCA)
4. **Klustring** - Gruppera liknande data utan fördefinierade kategorier

3. Förklara följande koncept

a) Syftet med att dela upp data i träningsdata, valideringsdata och testdata

- **Träningsdata:** Används för att lära modellen samband mellan features och target
- **Valideringsdata:** Används för att utvärdera modeller under utveckling och välja hyperparametrar
- **Testdata:** Används för slutlig, opartisk utvärdering av den valda modellen

Syfte: Förhindra överanpassning och få en realistisk bedömning av modellens prestanda på ny data.

b) K-delad korsvalidering

K-delad korsvalidering är en teknik där data delas upp i K lika stora delar (folds). Modellen tränas K gånger, varje gång med K-1 folds som träning och 1 fold som validering. Varje fold används exakt en gång som validering.

Fördelar: - Mer robust utvärdering - Bättre användning av begränsad data - Mindre beroende av en specifik datauppdelning

c) Root Mean Squared Error (RMSE)

RMSE är ett mått för att utvärdera regressionsmodeller. Det beräknas som kvadratroten av medelvärdet av de kvadrerade felen:

$$\text{RMSE} = \sqrt{(1/n \times \sum (y_i - \hat{y}_i)^2)}$$

Egenskaper: - Samma enhet som target-variabeln - Straffar stora fel mer än små fel (på grund av kvadrering) -
Lägre RMSE = bättre modell

d) Hyperparametrar vs Parametrar

- **Hyperparametrar:** Konfigureras före träning ("hur vi lär oss")
- Exempel: antal träd i Random Forest, learning rate, antal lager i neuralt nätverk
- Väljs av data scientist
- **Parametrar:** Lärs av modellen under träning ("vad vi lär oss")
- Exempel: koefficienter i linjär regression, vikter i neuralt nätverk
- Optimeras automatiskt av algoritmen

e) Grid Search

Grid Search är en metod för hyperparameteroptimering som systematiskt testar alla kombinationer av hyperparametrar i ett fördefinierat rutnät (grid).

Namnförklaring: - **"Grid"**: Hyperparametrarna bildar ett rutnät av kombinationer - **"Search"**: Algoritmen söker genom alla kombinationer för att hitta den bästa

refit=True: När GridSearchCV hittar bästa hyperparametrarna, tränar den om modellen på hela träningsdatan med dessa parametrar.

f) Kategorisk data och hantering

Kategorisk data är data som representerar kategorier eller grupper.

Typer: - **Nominal data:** Kategorier utan ordning (t.ex. färg: röd, blå, grön) - **Ordinal data:** Kategorier med ordning (t.ex. utbildningsnivå: grundskola, gymnasium, universitet)

Hanteringsmetoder: - **One-hot encoding:** Skapar binära kolumner för varje kategori - **Dummy-variable encoding:** Som one-hot men droppar en kategori för att undvika multicollinearity - **Ordinal encoding:** Mapper ordinala kategorier till numeriska värden

g) Feature Engineering

Feature Engineering är processen att skapa, välja och transformera variabler (features) för att förbättra modellens prestanda.

Exempel: - Skapa nya features från befintliga (t.ex. ålder från födelsedatum) - Kombinera features (t.ex. pris per kvadratmeter) - Transformera features (t.ex. logaritmisk transformation) - Hantera saknade värden

h) Principle of Parsimony

Principle of Parsimony (Ockhams rakkniv) säger att vi ska föredra enklare modeller framför komplexa modeller när de presterar lika bra.

Fördelar med enklare modeller: - Lättare att förstå och tolka - Mindre risk för överanpassning - Snabbare att träna och använda - Mer robusta

4. Vad menas med att "en modell är en förenkling av verkligheten"?

En modell är en förenkling eftersom den:

- **Väljer ut viktiga faktorer** och ignorerar mindre viktiga
- **Gör antaganden** om samband mellan variabler
- **Idealiserar verkligheten** för att göra den hanterbar
- **Fokuserar på specifika aspekter** av problemet

Exempel: En linjär modell antar linjära samband, men verkligheten är ofta mer komplex.

5. Vad menas med att en modell är "överanpassad" eller overfitted?

En överanpassad modell har lärt sig träningsdaten för väl, inklusive brus och slumpmässiga variationer. Den presterar bra på träningsdata men dåligt på ny data.

Symptom: - Hög precision på träningsdata - Låg precision på validering/testdata - Komplex modell som fångar brus

Lösningar: - Fler träningsdata - Regularisering - Enklare modellarkitektur

6. Högre är bättre i scikit-learn scoring, vad innebär det?

Scikit-learn förväntar sig att högre värden betyder bättre prestanda. För felmått som RMSE (där lägre är bättre) används därför negativa värden.

Exempel: - `neg_mean_squared_error` : Negativ MSE - `neg_root_mean_squared_error` : Negativ RMSE

Logik: -2.5 är "högre" än -3.0, så modellen med -2.5 presterar bättre.

7. Vad är tvärsnittsdata, tidsseriedata och paneldata?

Tvärsnittsdata: Observationer för olika individer vid en tidpunkt - Exempel: Inkomst för olika personer år 2023

Tidsseriedata: Observationer över tid för en individ - Exempel: Aktiepris för ett företag över 5 år

Paneldata: Observationer för olika individer över tid - Exempel: Inkomst för olika personer över 10 år

Resonemangsfrågor

8. Ge exempel på verkliga tillämpningsområden inom ML

Rekommendationssystem: - Netflix, Spotify, Amazon - Predikterar användarpreferenser

Bildklassificering: - Medicinsk diagnostik - Självkörande bilar - Säkerhetskameror

Naturligt språk: - Chatbots (ChatGPT) - Översättning - Sentimentanalys

Finans: - Kreditbedömning - Aktieprediktion - Bedrägeridetektering

E-handel: - Prisoptimering - Lagerhantering - Kundsegmentering

9. Förklara logiken bakom negativ mean squared error

Problemet: - MSE är ett felmått (lägre = bättre) - Scikit-learn förväntar sig "högre = bättre"

Lösningen: - Använd negativ MSE: `-MSE` - Nu gäller: högre (mindre negativt) = bättre

Exempel: - Modell A: $MSE = 2.5 \rightarrow \text{Score} = -2.5$ - Modell B: $MSE = 3.0 \rightarrow \text{Score} = -3.0$ - Modell A är bättre ($-2.5 > -3.0$)

Fördelar: - Konsekvent med scikit-learns konventioner - Enkel jämförelse mellan modeller - Fungerar med alla optimeringsalgoritmer

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 2: Ett ML projekt från början till slut

Faktafrågor

1. I kapitlet beskrivs en checklista med sju steg. Beskriv de sju stegen översiktligt. I verkligheten, följs dessa steg i en rak progression eller arbetar man generellt sett mer iterativt?

De sju stegen:

1. **Definiera problemet** - Vad vill vi lösa?
2. **Få tillgång till data** - Samla in relevant data
3. **Exploratory Data Analysis (EDA)** - Förstå datan
4. **Bearbeta datan** - Rensa och förbereda data
5. **ML-modellering** - Träna och utvärdera modeller
6. **Spara modellen** - För framtida användning
7. **Produktionssättning** - Använd modellen i verkligheten

Iterativt arbete:

- **Nej, man följer inte stegen i rak progression**
- **Iterativt arbete** är normen i verkligheten
- Man går tillbaka och justerar baserat på resultat
- EDA kan leda till att man behöver mer data (steg 2)
- Modellering kan visa att man behöver bättre dataförbehandling (steg 4)
- **Agile-approach** med snabba iterationer

2. Vad menas med att en modell produktionssätts?

Produktionssättning betyder att modellen används i verkligheten för att göra prediktioner på ny data.

Vad det innebär:

- **Modellen är live** och används av slutanvändare
- **Automatiserad prediktion** på ny data
- **Integration** med befintliga system
- **Skalbarhet** för att hantera många användare
- **Övervakning** av modellens prestanda

Exempel:

- **Netflix**: Rekommendationssystem som fungerar för alla användare
- **Bank**: Kreditbedömning som används för låneansökningar
- **E-handel**: Prisoptimering som fungerar i realtid

Utmaningar:

- **Data drift** - nya data kan skilja sig från träningsdata
- **Prestanda** - modellen måste vara snabb nog
- **Tillförlitlighet** - systemet måste vara stabilt
- **Underhåll** - modeller behöver uppdateras regelbundet

3. Vad är scikit-learn för något? Biblioteket följer några centrala designprinciper. Vilka är dessa? Vad är estimators, predictors och transformers?

Vad är scikit-learn?

- **Python-bibliotek** för maskininlärning
- **Open source** och gratis att använda
- **Omfattande** samling av ML-algoritmer
- **Användarvänligt** och väl dokumenterat

Centrala designprinciper:

1. **Konsistens** - Alla modeller följer samma interface
2. **Inspection** - Enkel tillgång till modellparametrar
3. **Non-proliferation of classes** - Få, väl definierade klasser
4. **Composition** - Modeller kan kombineras
5. **Sensible defaults** - Bra standardvärden

Estimators, Predictors och Transformers:

Estimators: - Objekt som kan lära sig från data - Har `.fit()` metod - Exempel: `LinearRegression()`, `RandomForestRegressor()`

Predictors: - Estimators som kan göra prediktioner - Har `.predict()` metod - Exempel: Alla regressions- och klassificeringsmodeller

Transformers: - Objekt som kan transformera data - Har `.fit()` och `.transform()` metoder - Exempel: `StandardScaler()`, `OneHotEncoder()`

4. Vad är TensorFlow och Keras?

TensorFlow:

- **Open source bibliotek** för maskininlärning
- **Utvecklat av Google**
- **Fokuserar på deep learning** och neurala nätverk
- **Stödjer både CPU och GPU**
- **Används för storskaliga ML-system**

Keras:

- **High-level API** för neurala nätverk
- **Bygger på TensorFlow** (eller andra backends)
- **Användarvänligt** och intuitivt
- **Snabb prototypning** av neurala nätverk

- **Modulär design** - enkelt att bygga komplexa modeller

Samband:

- **Keras är nu en del av TensorFlow**
- **TensorFlow.keras** är den officiella implementationen
- **Keras ger enklare interface** till TensorFlow's kraftfulla funktioner
- **TensorFlow hanterar low-level operationer** (GPU-optimering, etc.)

Resonemangsfrågor

5. Kalle och Stina diskuterar maskininlärning över en lunch. Kalle säger "om jag tränat en modell och den inte presterar bra nog på testdata så justerar jag den tills den gör det." Stina säger "det är ett stort fel att göra så, det enda du då åstadkommer är att du överanpassar testdata. Hela syftet med testdata försvinner då". Vad säger du om deras dialog?

Stina har helt rätt!

Vad Kalle gör fel:

- **Justerar modellen baserat på testdata**
- **Överanpassar testdata** (overfitting)
- **Förstör syftet med testdata**
- **Får optimistiska resultat** som inte representerar verklig prestanda

Varför testdata är viktig:

- **Oberoende utvärdering** av modellens prestanda
- **Simulerar verklig användning** på ny data
- **Hjälper att upptäcka overfitting**
- **Ger en ärlig bedömning** av modellens kvalitet

Rätt process:

1. **Träna modell** på träningsdata
2. **Justera hyperparametrar** baserat på valideringsdata
3. **Utvärdera EN gång** på testdata
4. **Testdata används aldrig för justering**

Kalles misstag:

- **Data leakage** - information från testdata läcker in i modellen
- **Optimistisk bias** - modellen presterar bättre än den egentligen gör
- **Förlorad tillförlitlighet** - ingen ärlig bedömning av prestanda

6. Många AI/ML projekt uppnår inte de ursprungligen satta målen eller att ens passera någon form av prototyp-stadie. Vad tror du detta beror på och hur ska vi förhålla oss till det?

Vanliga orsaker till misslyckande:

1. Orealistiska förväntningar:

- **"AI löser allt"** - för höga förväntningar
- **Underestimera komplexitet** - ML är inte magi
- **Orealistiska tidsramar** - ML-projekt tar tid

2. Data-problem:

- **Dålig datakvalitet** - skräp in, skräp ut
- **Otillräcklig data** - för lite träningsdata
- **Saknade värden** - ofullständig data
- **Data bias** - fördomar i datan

3. Tekniska utmaningar:

- **Komplexitet** - för avancerade modeller
- **Skalbarhet** - systemet kan inte hantera belastning
- **Integration** - svårt att integrera med befintliga system

4. Organisatoriska problem:

- **Lack of expertise** - ingen ML-kompetens
- **Motstånd till förändring** - organisationen är inte redo
- **Dålig kommunikation** - missförstånd mellan team

Hur vi ska förhålla oss till det:

- **Acceptera att misslyckanden händer** - det är normalt
- **Lär av misstagen** - varje misslyckande ger insikter
- **Starta smått** - prototyper innan storskaliga projekt
- **Sätt realistiska mål** - gradvis förbättring
- **Fokusera på värde** - lösa verkliga problem

8. Förklara vad koden nedan gör. Varför är det viktigt att kunna spara en modell?

Vad koden gör:

Koden demonstrerar hur man sparar och laddar en tränad modell:

1. **Skapar syntetisk data** med `make_regression()`
2. **Tränar en linjär regressionsmodell** på datan
3. **Sparar modellen** till fil med `dump()` från joblib
4. **Läser tillbaka modellen** från fil med `load()`
5. **Använder den laddade modellen** för att göra prediktioner

Varför är det viktigt att spara modeller?

1. **Vi vill inte träna om modellen varje gång** - träning kan ta lång tid
2. **Modeller kan vara dyra att träna** - kostar tid och resurser
3. **Vi vill kunna använda modellen i produktion** - modellen måste vara tillgänglig
4. **Vi vill dela modellen med andra** - kollegor eller system kan använda den

5. **Versionering** - spara olika versioner av modeller för jämförelse
6. **Reproducerbarhet** - samma modell kan användas flera gånger
7. **Backup** - säkerhetskopiera tränade modeller

Praktiska fördelar:

- **Snabbare deployment** - ingen behov av att träna om
 - **Konsistens** - samma modell används överallt
 - **Resurseffektivitet** - sparar beräkningskraft
 - **Flexibilitet** - enkelt att byta mellan olika modeller
-

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 3: Regression

Faktafrågor

1. Vad kännetecknar regressionsproblem? Ge några exempel på tillämpningsområden.

Regressionsproblem kännetecknas av att vi försöker förutsäga en **kontinuerlig numerisk variabel** (target/output) baserat på en eller flera oberoende variabler (features/inputs).

Huvudkännetecken:

- **Kontinuerlig output:** Target-variabeln kan anta alla värden inom ett intervall
- **Numerisk prediktion:** Vi försöker förutsäga ett specifikt numeriskt värde
- **Kvantitativ analys:** Resultatet är ett mätbart värde

Exempel på tillämpningsområden:

1. **Ekonomi och Finans**
 2. Lönsprediktion baserat på ålder, utbildning, erfarenhet
 3. Aktieprisprediktion baserat på ekonomiska indikatorer
 4. Försäljningsprognoser baserat på marknadsdata
5. **Fastigheter**
 6. Husprisprediktion baserat på storlek, ålder, plats
 7. Hyresprisprediktion baserat på läge och faciliteter
8. **Transport och Logistik**
 9. Bilprisprediktion baserat på miltal, ålder, märke
 10. Drivmedelsförbrukning baserat på körstil och väglag
 11. Leveranstidsprediktion baserat på avstånd och trafik
12. **Medicin och Hälsa**
 13. Diabetes progression baserat på blodvärden
 14. Blodtrycksprediktion baserat på livsstil
 15. Medicindosering baserat på patientdata
16. **Jordbruk och Miljö**
 17. Skördeprediktion baserat på väder och gödning
 18. Temperaturprediktion baserat på historisk data
 19. Luftkvalitetsprediktion baserat på utsläpp

20. Industri och Produktion

- 21. Produktionsvolym baserat på resurser
- 22. Underhållsbehov baserat på maskindata
- 23. Kvalitetsmått baserat på processparametrar

Viktigt att komma ihåg: Regression handlar alltid om att förutsäga ett **kontinuerligt värde**, till skillnad från klassificering som förutsäger kategorier.

2. Förklara utvärderingsmått RMSE, MSE och MAE.

RMSE (Root Mean Square Error)

Formel: $RMSE = \sqrt{1/n \times \sum (y_{true} - y_{pred})^2}$

Vad det mäter:

- Kvadratroten av genomsnittliga kvadratiska felet
- Ger fel i samma enhet som target-variabeln

Fördelar:

- Straffar stora fel hårdare (kvadrering)
- Samma enhet som target (tolkbart)
- Standardmått inom regression

Exempel: Om vi predikerar lön i kronor, blir RMSE också i kronor

MSE (Mean Square Error)

Formel: $MSE = 1/n \times \sum (y_{true} - y_{pred})^2$

Vad det mäter:

- Genomsnittliga kvadratiska felet
- Samma som RMSE men utan kvadratroten

Fördelar:

- Billigare att beräkna än RMSE
- Samma rangordning som RMSE
- Matematiskt enklare att optimera

Exempel: Om $RMSE = 1000$ kr, så är $MSE = 1,000,000$ kr²

MAE (Mean Absolute Error)

Formel: $MAE = 1/n \times \sum |y_{true} - y_{pred}|$

Vad det mäter:

- Genomsnittliga absoluta felet
- Alla fel värderas linjärt

Fördelar:

- Mindre känsligt för extremvärden
- Samma enhet som target
- Robust mot outliers

Exempel: Mindre känsligt för stora fel än RMSE

Jämförelse RMSE vs MAE:

Exempel:

- Modell 1: Fel på 10 och 10 $\rightarrow$ RMSE = 10, MAE = 10
- Modell 2: Fel på 5 och 15 $\rightarrow$ RMSE = 11.2, MAE = 10

Slutsats:

- RMSE föredrar Modell 1 (jämnare fel)
- MAE är neutral (samma totala fel)

Praktisk regel:

- Använd **RMSE** när stora fel är värre än många små fel
- Använd **MAE** när alla fel ska värderas lika mycket

3. Om vi ska rangordna olika modeller, spelar det någon roll om RMSE eller MSE används? Varför?

Nej, det spelar **INGEN** roll för rangordningen!

Varför?

Matematisk förklaring:

- $RMSE = \sqrt{MSE}$
- Kvadratroten är en **monoton funktion**
- Det betyder att om $A > B$, så är $\sqrt{A} > \sqrt{B}$

Praktiskt exempel:

```
Modell A: MSE = 100, RMSE = 10
Modell B: MSE = 144, RMSE = 12
Modell C: MSE = 81,  RMSE = 9
```

Rangordning med MSE: C (81) < A (100) < B (144) **Rangordning med RMSE:** C (9) < A (10) < B (12)

Resultat: Samma rangordning!

När väljer man vad?

Använd MSE när:

- Du bara vill rangordna modeller
- Beräkningshastighet är viktigt
- Du optimerar hyperparametrar

Använd RMSE när:

- Du vill tolka felstorleken
- Du presenterar resultat för andra
- Du vill förstå praktisk betydelse

Praktisk regel:

1. **Under utveckling:** Använd MSE (snabbare)
2. **Vid presentation:** Använd RMSE (tolkbart)
3. **Rangordning:** Spelar ingen roll vilket du väljer

Viktigt: Detta gäller bara för MSE vs RMSE. MAE kan ge annan rangordning!

4. Förklara mycket översiktligt vad gradient descent är.

Gradient Descent - Optimeringsalgoritm

Enkelt koncept: Gradient descent är en algoritm som hittar **minimum** av en funktion genom att gå **nedåt längs** lutningen.

Analogier för förståelse:

Kulle-analogin:

- Tänk dig att du står på en kulle
- Du vill komma till botten (minimum)
- Du går i den riktning som är nedåt (negativ lutning)
- Du fortsätter tills du når botten

Termostat-analogin:

- Du vill hitta den perfekta temperaturen
- Om det är för varmt, sänker du temperaturen
- Om det är för kallt, höjer du temperaturen
- Du justerar tills det är perfekt

I maskininlärning:

Syfte: Hitta de bästa parametrarna för en modell

Process:

1. **Starta** med slumpmässiga parametrar
2. **Beräkna** hur fel modellen gör (loss function)
3. **Beräkna** lutningen (gradient) av felet
4. **Uppdatera** parametrarna i motsatt riktning av lutningen
5. **Upprepa** tills felet inte minskar mer

Viktiga koncept:

Learning Rate (α):

- Hur stora steg du tar
- För stort: kan hoppa över minimum
- För litet: tar lång tid att konvergera

Konvergens:

- När algoritmen hittar minimum
- Lutningen blir nära noll
- Modellen är tränad

Varianter:

1. Batch Gradient Descent:

2. Använder all data för varje steg
3. Stabil men långsam

4. Stochastic Gradient Descent:

5. Använder en observation i taget
6. Snabb men brusig

7. Mini-batch Gradient Descent:

8. Använder små grupper av data
9. Kompromiss mellan hastighet och stabilitet

Praktisk betydelse:

- **Automatisk träning:** Datorn hittar bästa parametrarna
- **Skalbarhet:** Fungerar för stora dataset
- **Flexibilitet:** Kan användas för många olika modeller

Viktigt: Du behöver inte implementera gradient descent själv - scikit-learn gör det åt dig!

5. Vad är the bias variance trade-off? Varför är mer komplexa modeller inte alltid bättre?

Bias-Variance Trade-off

Grundkoncept: Det finns en balans mellan modellens **bias** och **variance** som påverkar dess prestanda.

Vad är Bias och Variance?

Bias (Fördom):

- Fel från felaktiga antaganden om data
- Modellen är för enkel för att fånga sambanden
- **Underanpassning (Underfitting)**

- Exempel: Linjär modell för icke-linjära data

Variance (Variation):

- Fel från känslighet för träningsdata
- Modellen är för komplex och anpassar sig för mycket
- **Överanpassning (Overfitting)**
- Exempel: Polynom av hög grad som följer brus

Trade-off-förhållandet:

Enkel modell:	Hög Bias + Låg Variance
Komplex modell:	Låg Bias + Hög Variance

Regel: När du minskar bias, ökar variance och vice versa.

Varför är komplexa modeller inte alltid bättre?

1. Överanpassning (Overfitting):

- Komplexa modeller kan lära sig brus i träningsdata
- De generaliserar dåligt till ny data
- Högre variance leder till sämre prestanda på testdata

2. Beräkningskostnad:

- Komplexa modeller tar längre tid att träna
- Kräver mer minne och processorkraft
- Dyrare att använda i produktion

3. Tolkningsbarhet:

- Komplexa modeller är svåra att förstå
- Svårt att förklara varför de gör vissa prediktioner
- Mindre användbara för beslutstagning

4. Instabilitet:

- Små ändringar i data kan ge stora ändringar i modellen
- Mindre robusta och pålitliga

Praktiskt exempel:

Linjär regression (enkel):

- Hög bias: Kan inte fånga icke-linjära samband
- Låg variance: Stabil och robust

Polynomregression (komplex):

- Låg bias: Kan fånga komplexa samband
- Hög variance: Känslig för brus och outliers

Hur hittar man rätt balans?

1. Korsvalidering:

- Testa olika modellkomplexitet
- Hitta sweet spot mellan bias och variance

2. Regularisering:

- Ridge, Lasso, Elastic Net
- Minska variance genom att begränsa modellen

3. Ensemble-metoder:

- Random Forest, Bagging
- Kombinera flera modeller för bättre balans

Praktisk regel:

"Börja enkelt, gör komplexare endast om nödvändigt"

1. Börja med linjär regression
2. Om prestanda är dålig, prova mer komplexa modeller
3. Använd korsvalidering för att hitta optimal komplexitet
4. Kom ihåg: Enkel modell som fungerar är bättre än komplex modell som inte fungerar!

6. Några vanligt förekommande modeller för regressionsproblem är enligt nedan. Förklara översiktligt hur respektive modell fungerar.

a) Linjär regression

Princip: Modellerar ett linjärt samband mellan features och target.

Formel: $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p$

Fördelar:

- Enkel och tolkbar
- Snabb att träna och använda
- Bra utgångspunkt

Nackdelar:

- Antar linjära samband
- Kan inte fånga komplexa mönster

Användning: Baseline-modell, när sambanden är linjära

b) Ridge regression (L2-regularisering)

Princip: Linjär regression med straff för stora koefficienter.

Formel: $\text{MSE} + \alpha \times \sum \theta_i^2$

Effekt:

- Krymper koefficienter mot 0
- Förhindrar överanpassning
- Minskar variance

Användning: När du har många features eller överanpassning

c) Lasso regression (L1-regularisering)

Princip: Linjär regression med straff för absoluta koefficienter.

Formel: $MSE + \alpha \times \sum |\theta_i|$

Effekt:

- Sätter vissa koefficienter till exakt 0
- Automatisk variabelselektion
- Sparar bara viktiga features

Användning: När du vill välja ut viktiga features

d) Elastic Net

Princip: Kombination av Ridge och Lasso.

Formel: $MSE + r \times \alpha \times \sum |\theta_i| + (1-r)/2 \times \alpha \times \sum \theta_i^2$

Fördelar:

- Bästa av båda världarna
- Hanterar korrelerade features
- Flexibel regularisering

Användning: När du vill ha både variabelselektion och regularisering

e) Support Vector Machines (SVM)

Princip: Skapar en "väg" runt data med given bredd.

Funktion:

- Skapar en väg med bredd ϵ
- Innehåller så många observationer som möjligt
- Mitten-linjen blir prediktionen

Fördelar:

- Robust mot outliers
- Bra för små dataset
- Kan hantera icke-linjära samband (med kernels)

Nackdelar:

- Långsam för stora dataset
- Känslig för hyperparametrar

f) Beslutsträd

Princip: Delar data upp i regioner med ja/nej-frågor.

Funktion:

- Trädstruktur med noder och grenar
- Varje nod ställer en fråga om en feature
- Lövnoder ger prediktioner

Fördelar:

- Tolkbart och intuitiv
- Hanterar icke-linjära samband
- Kan hantera kategoriska features

Nackdelar:

- Instabil (små ändringar kan ge stora skillnader)
- Risk för överanpassning

g) Ensemble learning

Princip: Kombinerar flera modeller för bättre prestanda.

Voting Regression:

- Flera modeller röstar
- Tar medelvärde av prediktioner

Bagging:

- Skapar flera dataset med återläggning
- Tränar modeller på varje dataset
- Kombinerar prediktioner

Fördelar:

- Minskar variance
- Stabilare prediktioner
- Bättre generalisering

h) Random Forest

Princip: Ensemble av beslutsträd med bagging och slumpmässig variabelselektion.

Funktion:

- Skapar många beslutsträd
- Varje träd använder slumpmässig del av data och features
- Kombinerar prediktioner från alla träd

Fördelar:

- Mycket robust
- Bra prestanda
- Mindre risk för överanpassning
- Kan hantera saknade värden

Nackdelar:

- Svår att tolka
- Kan vara långsam
- Kräver mer minne

7. Vad menas med white box modeller och black box modeller?

White Box Modeller

Definition: Modeller där det är **enkelt att förstå och förklara** hur de kommer fram till sina prediktioner.

Kännetecken:

- **Tolkbara:** Du kan se exakt hur modellen fungerar
- **Transparenta:** Steg-för-steg förklaring möjlig
- **Intuitiva:** Lätt att förstå logiken

Exempel:

- **Linjär regression:** Du kan se koefficienterna och förstå vilken påverkan varje feature har
- **Beslutsträd:** Du kan följa vägen genom trädet och se varför en prediktion gjordes
- **Logistisk regression:** Du kan se odds-ratio och förstå sannolikheter

Fördelar:

- **Förtroende:** Användare förstår hur modellen fungerar
- **Felsökning:** Lätt att hitta problem
- **Regulering:** Uppfyller krav på förklarbarhet
- **Beslutstagning:** Stödjer mänskliga beslut

Nackdelar:

- Kan ha sämre prestanda än komplexa modeller
- Begränsad kapacitet att fånga komplexa samband

Black Box Modeller

Definition: Modeller där det är **svårt eller omöjligt att förstå** hur de kommer fram till sina prediktioner.

Kännetecken:

- **Ogenomträngliga:** Svårt att se inuti modellen
- **Komplexa:** Många lager och interaktioner
- **Abstrakta:** Logiken är inte direkt synlig

Exempel:

- **Neurala nätverk:** Många lager med komplexa vikter
- **Random Forest:** Många träd med komplexa interaktioner
- **SVM med kernels:** Transformationer som är svåra att visualisera
- **Ensemble-metoder:** Kombination av många modeller

Fördelar:

- **Hög prestanda:** Kan fånga komplexa samband
- **Flexibilitet:** Kan hantera många olika datatyper
- **Skalbarhet:** Fungerar bra med stora dataset

Nackdelar:

- **Lågt förtroende:** Användare förstår inte hur modellen fungerar
- **Svår felsökning:** Problem är svåra att identifiera
- **Regulatoriska problem:** Kan bryta mot krav på förklarbarhet
- **Etiska problem:** Kan göra diskriminerande beslut utan förklaring

Praktiska överväganden:**När väljer man White Box?**

- Medicinska beslut
- Finansiella beslut
- Juridiska beslut
- När förklarbarhet är viktigare än prestanda

När väljer man Black Box?

- Rekommendationssystem
- Bildklassificering
- När prestanda är viktigare än förklarbarhet
- När modellen är komplext men fungerar bra

Framtida trender:**Explainable AI (XAI):**

- Försök att göra black box-modeller mer förklarbara
- Tekniker som SHAP, LIME
- Balans mellan prestanda och förklarbarhet

Hybrid-modeller:

- Kombinera white box och black box
- Få bästa av båda världarna
- Ökande trend inom AI-forskning

8. Vad är skillnaden mellan bagging och pasting?

Bagging (Bootstrap Aggregating)

Princip: Skapar nya dataset genom **slumpmässigt urval med återläggning**.

Process:

1. Ta ett slumpmässigt urval från originaldata
2. **Samma observation kan väljas flera gånger** (återläggning)
3. Skapa ett nytt dataset med samma storlek som original
4. Träna en modell på det nya datasetet
5. Upprepa för flera modeller
6. Kombinera prediktioner (t.ex. medelvärde)

Egenskaper:

- **Med återläggning:** Samma observation kan förekomma flera gånger
- **Dataset-storlek:** Samma som original
- **Överlapp:** Många observationer förekommer i flera dataset
- **Variation:** Mindre variation mellan dataset

Fördelar:

- **Stabilare:** Mindre variation mellan modeller
- **Robust:** Fungerar bra med många modeller
- **Bevisad metod:** Väldokumenterad och testad

Nackdelar:

- **Mindre variation:** Dataset är mer lika varandra
- **Risk för överanpassning:** Om originaldata är liten

Pasting

Princip: Skapar nya dataset genom **slumpmässigt urval utan återläggning**.

Process:

1. Ta ett slumpmässigt urval från originaldata
2. **Varje observation kan bara väljas en gång** (utan återläggning)
3. Skapa ett nytt dataset (ofta mindre än original)
4. Träna en modell på det nya datasetet
5. Upprepa för flera modeller
6. Kombinera prediktioner

Egenskaper:

- **Utan återläggning:** Varje observation kan bara väljas en gång
- **Dataset-storlek:** Ofta mindre än original
- **Överlapp:** Mindre överlapp mellan dataset
- **Variation:** Större variation mellan dataset

Fördelar:

- **Större variation:** Dataset är mer olika varandra
- **Mindre överanpassning:** Särskilt bra för stora dataset
- **Snabbare träning:** Mindre dataset = snabbare träning

Nackdelar:

- **Mindre stabil:** Större variation mellan modeller
- **Kräver stora dataset:** Fungerar bäst med mycket data

Jämförelse:

Egenskap	Bagging	Pasting
Urval	Med återläggning	Utan återläggning
Dataset-storlek	Samma som original	Mindre än original
Överlapp	Hög	Låg
Variation	Låg	Hög
Stabilitet	Hög	Låg
Prestanda	Bra för många modeller	Bra för stora dataset

När väljer man vad?

Använd Bagging när:

- Du har relativt lite data
- Du vill ha stabila resultat
- Du använder många modeller
- Du vill minimera variation

Använd Pasting när:

- Du har mycket data
- Du vill ha större variation mellan modeller
- Träningshastighet är viktigt
- Du vill undvika överanpassning

Praktiskt exempel:

Bagging (Random Forest):

- Varje träd tränas på ett dataset med återläggning
- Samma observationer kan förekomma i flera träd
- Ger stabila och robusta resultat

Pasting:

- Varje modell tränas på en unik del av data
- Inga observationer delas mellan modeller
- Ger större variation men kan vara mindre stabil

Sammanfattning:

Bagging är som att dra kort från en kortlek och lägga tillbaka dem - samma kort kan dras flera gånger.

Pasting är som att dela ut kort från en kortlek - varje kort kan bara delas ut en gång.

Båda metoderna har sina fördelar och används i olika situationer beroende på datamängd och mål.

Resonemangsfrågor

9. Förklara hur man kan tolka figur 3.1 på sidan 113.

Figur 3.1 - Enkel linjär regression

Vad visar figuren: Figur 3.1 visar en **schematisk bild över enkel linjär regression** med data, regressionslinje och residualer.

Komponenter i figuren:

1. Svarta punkter (Data):

- Representerar **observerade datapunkter** (y_i)
- Varje punkt är en observation med x-värde och y-värde
- Dessa är de verkliga värdena från datasetet

2. Blå linje (Regressionslinje):

- Representerar **predikterade värden** ($\hat{y} = \theta_0 + \theta_1 x$)
- Visar vad modellen förutsäger för varje x-värde
- Den bästa linjära approximationen av data

3. Röda vertikala linjer (Residualer):

- Representerar **fel** mellan observerade och predikterade värden
- $e_i = y_i - \hat{y}_i$ (observerat - predikterat)
- Visar hur mycket modellen missar för varje datapunkt

Tolkning av regressionslinjen:

Intercept (θ_0):

- Var linjen skär y-axeln
- Predikterat värde när $x = 0$
- I praktiken: Baslinje-värde

Lutning (θ_1):

- Hur mycket y ökar när x ökar med 1 enhet
- Positiv lutning: Positivt samband
- Negativ lutning: Negativt samband
- Större lutning: Starkare samband

Tolkning av residualerna:

Stora residualer:

- Modellen missar mycket för dessa punkter
- Kan indikera outliers eller icke-linjära samband
- Punkter som är svåra att förutsäga

Små residualer:

- Modellen förutsäger väl för dessa punkter
- Punkter som följer det linjära sambandet
- Bra anpassning av modellen

Mönster i residualerna:

- Om residualerna följer ett mönster: Modellen kanske inte är lämplig
- Om residualerna är slumpmässiga: Bra anpassning

Praktisk tolkning:**Exempel - Lön vs Ålder:**

- **Svarta punkter:** Verkliga löner för olika åldrar
- **Blå linje:** Predikterad lön baserat på ålder
- **Röda linjer:** Hur mycket modellen missar för varje person

Insikter:

- Positiv lutning: Lön ökar med ålder
- Stora residualer: Vissa personer tjänar mycket mer/mindre än förväntat
- Små residualer: Modellen förutsäger lön väl för de flesta

Kvalitetsbedömning:**Bra anpassning:**

- Punkterna ligger nära linjen
- Små residualer
- Slumpmässiga residualer

Dålig anpassning:

- Punkterna spridda långt från linjen
- Stora residualer
- Mönster i residualerna

Begränsningar:**Enkel linjär regression antar:**

- Linjärt samband mellan x och y
- Oberoende observationer
- Konstant varians (homoskedasticitet)
- Normalfördelade residualer

Om dessa antaganden inte uppfylls:

- Modellen kanske inte är lämplig
- Andra modeller kan vara bättre
- Transformationer kan behövas

Sammanfattning:

Figur 3.1 är en **grundläggande visualisering** som visar:

1. **Verkliga data** (svarta punkter)
2. **Modellens prediktioner** (blå linje)
3. **Modellens fel** (röda linjer)

Den hjälper oss att förstå hur väl modellen anpassar sig till data och var den missar.

10. Förklara hur man kan tolka figur 3.13 på sidan 140. Hur hänger den ihop med figur 3.14 på sidan 141?

Figur 3.13 - Beslutsträd visualisering

Vad visar figuren: Figur 3.13 visar en **trädstruktur** med noder, grenar och beslut som beslutsträdet använder för att göra prediktioner.

Komponenter i figur 3.13:

1. Rotnod (översta noden):

- Startpunkt för prediktionen
- Innehåller första beslutet
- Exempel: "Är $x_2 \leq 0.438$?"

2. Interna noder:

- Noder som ställer frågor
- Delar data upp i mindre grupper
- Exempel: "Är $x_1 \leq -0.654$?"

3. Lövnoder (yttersta noderna):

- Slutpunkter för prediktionen
- Innehåller det slutliga predikterade värdet
- Exempel: "value = 1.837"

4. Information i varje nod:

- **Samples:** Antal observationer i noden
- **Squared error:** MSE för noden
- **Value:** Predikterat värde (endast i lövnoder)

Hur prediktion fungerar:

Steg-för-steg process:

1. **Starta i rotnoden**
2. **Ställ frågan** (t.ex. "Är $x_2 \leq 0.438$?")
3. **Gå till vänster** om svaret är JA
4. **Gå till höger** om svaret är NEJ
5. **Upprepa** tills du når en lövnod
6. **Prediktera** värdet från lövnoden

Exempel:

- Observation: $x_1 = -1$, $x_2 = 0.55$
- Fråga 1: "Är $x_2 \leq 0.438$?" → NEJ → Gå höger
- Fråga 2: "Är $x_1 \leq -0.654$?" → JA → Gå vänster
- Resultat: Lövnod med value = 1.837

Figur 3.14 - Beslutsträd i 2D-rum

Vad visar figuren: Figur 3.14 visar **samma beslutsträd** men visualiserat i det 2-dimensionella rummet med x_1 och x_2 som axlar.

Komponenter i figur 3.14:

1. Datapunkter:

- Svarta punkter som representerar observationer
- Varje punkt har koordinater (x_1 , x_2)
- Färgkodning visar verkliga y-värden

2. Beslutsgränser:

- Vertikala och horisontella linjer
- Delar upp rummet i regioner
- Motsvarar besluten i figur 3.13

3. Regioner:

- Olika färgade områden
- Varje region motsvarar en lövnod
- Alla punkter i samma region får samma prediktion

4. Färgskala:

- Visar verkliga y-värden
- Hjälper att förstå datafördelningen
- Jämför med predikterade värden

Samband mellan figurerna:

Korrespondens:

- Rotnod i 3.13 → Första beslutsgränsen i 3.14
- Interna noder i 3.13 → Ytterligare beslutsgränser i 3.14

- Lövnoder i 3.13 → Regioner i 3.14
- Predikterade värden i 3.13 → Färgade regioner i 3.14

Exempel-korrespondens:

- Beslut " $x_2 \leq 0.438$ " i 3.13 → Horisontell linje vid $x_2 = 0.438$ i 3.14
- Beslut " $x_1 \leq -0.654$ " i 3.13 → Vertikal linje vid $x_1 = -0.654$ i 3.14
- Lövnod med value = 1.837 i 3.13 → Region med motsvarande färg i 3.14

Insikter från visualiseringen:

Trädstruktur (3.13):

- Visar **logisk struktur** av besluten
- Lätt att följa prediktionsprocessen
- Visar hierarki av beslut

2D-visualisering (3.14):

- Visar **geometrisk struktur** av besluten
- Lätt att se hur data är uppdelad
- Visar komplexitet och anpassning

Praktisk tolkning:

Fördelar med beslutsträd:

- **Tolkbara:** Du kan se exakt varför en prediktion gjordes
- **Flexibla:** Kan hantera icke-linjära samband
- **Automatiska:** Väljer bästa beslut automatiskt

Begränsningar:

- **Stuckade beslut:** Kan bara dela upp rummet med vertikala/horizontella linjer
- **Instabila:** Små ändringar i data kan ge helt annorlunda träd
- **Överanpassning:** Kan bli för komplexa

Sammanfattning:

Figur 3.13 visar beslutsträdet som en **logisk struktur** med noder och beslut.

Figur 3.14 visar samma beslutsträd som en **geometrisk uppdelning** av datarummet.

Tillsammans ger de en komplett förståelse för hur beslutsträd fungerar både logiskt och geometriskt.

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 4: Klassificering

Faktafrågor

1. Vad kännetecknar klassificeringsproblem? Ge några exempel på tillämpningsområden.

Klassificeringsproblem kännetecknas av:

- **Diskret output:** Modellen försöker förutsäga vilken kategori/klass en observation tillhör
- **Finit antal klasser:** Begränsat antal möjliga utfall
- **Kvalitativ prediktion:** Resultatet är en kategori, inte ett numeriskt värde

Exempel på tillämpningsområden:

- **Spam-detektering:** Klassificera e-post som spam eller icke-spam
- **Medicinsk diagnostik:** Identifiera sjukdomar från röntgenbilder
- **Kreditrisk:** Bedöma om en person ska få lån eller inte
- **Bildklassificering:** Känna igen objekt i bilder (katt/hund)
- **Ansiktsgenkänning:** Identifiera personer från foton
- **Kundsegmentering:** Kategorisera kunder baserat på beteende

2. Förklara hur OvR- och OvO-algoritmerna fungerar.

One-vs-Rest (OvR) - "En mot Alla":

- Tränar en binär modell per klass
- Varje modell lär sig att skilja en klass från alla andra
- För k klasser tränas k modeller
- Prediktion: Välj klass med högst sannolikhet

Exempel för 3 klasser (Äpple, Banan, Apelsin):

- Modell 1: Äpple vs (Banan + Apelsin)
- Modell 2: Banan vs (Äpple + Apelsin)
- Modell 3: Apelsin vs (Äpple + Banan)

One-vs-One (OvO) - "En mot En":

- Tränar en binär modell för varje par av klasser
- För k klasser tränas $k(k-1)/2$ modeller
- Prediktion: Röstning mellan alla modeller

Exempel för 3 klasser:

- Modell 1: Äpple vs Banan
- Modell 2: Äpple vs Apelsin
- Modell 3: Banan vs Apelsin

3. Förklara följande utvärderingsmått:

a) Confusion Matrix

En tabell som visar hur modellen presterade på varje klass:

		Predikterad	
		Positiv	Negativ
Faktisk	Positiv	TP	FN
Faktisk	Negativ	FP	TN

- **TP (True Positive):** Rätt identifierade positiva
- **TN (True Negative):** Rätt identifierade negativa
- **FP (False Positive):** Felaktigt flaggade som positiva
- **FN (False Negative):** Missade positiva

b) Accuracy

Formel: $(TP + TN) / (TP + TN + FP + FN)$

Andel korrekta prediktioner av alla prediktioner.

c) Precision

Formel: $TP / (TP + FP)$

"När modellen säger 'positiv', hur ofta har den rätt?"

d) Recall

Formel: $TP / (TP + FN)$

"Av alla som var positiva, hur många hittade modellen?"

Recall mäter andelen **positiva exempel** som modellen lyckades identifiera korrekt.

e) F1-Score

Formel: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Harmoniskt medelvärde som balanserar precision och recall.

f) ROC-kurvan

- **X-axel:** False Positive Rate ($FP / (FP + TN)$)
- **Y-axel:** True Positive Rate ($TP / (TP + FN)$)
- Visar trade-off mellan TPR och FPR vid olika tröskelvärden
- AUC (Area Under Curve) ger ett mått på modellens prestanda

4. Vad är precision-recall tradeoff för något?

Precision-recall tradeoff är det förhållande att:

- **Högre precision** ofta leder till **lägre recall**
- **Högre recall** ofta leder till **lägre precision**

Exempel:

- Om vi sänker tröskelvärde för att flagga spam:
- Vi hittar fler spam (högre recall)
- Men vi flaggar också mer legitim e-post som spam (lägre precision)

Praktiska implikationer:

- **Spam-filter:** Högre precision (vill inte missa viktig e-post)
- **Cancer-screening:** Högre recall (vill inte missa någon cancer)

5. Några vanligt förekommande modeller för klassificeringsproblem är enligt nedan. Förklara översiktligt hur respektive modell fungerar.

a) Logistisk Regression

- **Grundprinciper:** Linjär modell med sigmoid-funktion
- **Output:** Sannolikhet mellan 0 och 1
- **Fördelar:** Tolkbart, snabbt, bra baseline
- **Nackdelar:** Antar linjäritet

b) Support Vector Machines (SVM)

- **Grundprinciper:** Hittar optimalt separerande hyperplan
- **Kernel-trick:** Mapperar data till högre dimension
- **Fördelar:** Effektiv i höga dimensioner, robust
- **Nackdelar:** Känslig för skalning

c) Beslutsträd

- **Grundprinciper:** Hierarkisk struktur med noder och kanter
- **Träning:** Väljer bästa feature att dela på (Information Gain)
- **Fördelar:** Mycket tolkbart, hanterar olika datatyper
- **Nackdelar:** Instabil, känslig för overfitting

d) Ensemble Learning

- **Grundprinciper:** Kombinerar flera modeller
- **Bagging:** Bootstrap Aggregating (Random Forest)
- **Boosting:** Sekventiell träning (AdaBoost, XGBoost)
- **Voting:** Majoritetsröstning eller vägt genomsnitt

e) Random Forest

- **Grundprinciper:** Ensemble av besluts-träd med bagging
- **Feature subsampling:** Väljer slumpmässigt subset av features

- **Fördelar:** Robust, ger feature importance
- **Nackdelar:** Mindre tolkbar än enskilda träd

f) Extra Trees

- **Grundprinciper:** Liknande Random Forest men med mer randomisering
- **Skillnad:** Väljer tröskelvärden slumpmässigt istället för optimalt
- **Fördelar:** Snabbare träning, mindre overfitting
- **Nackdelar:** Kan ha lägre prestanda

6. Vad innebär det att vi kan kolla på feature importance med hjälp av trädmodeller såsom beslutsträd eller random forest?

Feature importance visar hur viktig varje feature är för modellens prediktioner:

Beräkningsmetoder:

- **Information Gain:** Hur mycket information varje feature bidrar med
- **Gini Importance:** Baserat på Gini-index för varje feature
- **Permutation Importance:** Hur mycket prestandan sjunker när feature-värden blandas om

Praktisk användning:

- **Feature selection:** Ta bort irrelevanta features
- **Tolkbarhet:** Förstå vad som påverkar prediktionerna
- **Domain knowledge:** Validera mot expertis
- **Kostnadsbesparing:** Fokusera på viktiga features

Exempel: I en modell för kreditrisk kan feature importance visa att:

- Inkomst är den viktigaste featuren (40%)
- Kredithistorik är näst viktigast (30%)
- Ålder har mindre betydelse (10%)

Resonemangsfrågor

7. Stina säger till Kalle på lunchsamtalet "jag vill ha högsta möjliga precision för vår klassificeringsmodell". Kalle funderar ett tag och säger "men vad händer då med recall"? Vad hade du svarat? I vilka fall kan man tänka sig vilja ha en så hög precision som möjligt? I vilka fall kan det vara dåligt? Om vi tänker oss rättsväsendet där en slutgiltig dom kan leda till fängelse, vad kan vi då säga om precision-recall tradeoff?

Kalles oro är berättigad: När vi ökar precision (genom att höja tröskelvärdet) så får vi ofta lägre recall. Detta är precision-recall tradeoff.

När hög precision är bra:

- **Spam-filter:** Du vill inte missa viktig e-post
- **Kvalitetskontroll:** Du vill inte kassera bra produkter
- **Rekommendationssystem:** Du vill bara rekommendera relevanta produkter

- **Medicinsk screening:** Du vill inte ge behandling till friska personer

När hög precision kan vara dåligt:

- **Cancer-screening:** Du missar många cancerfall (låg recall)
- **Säkerhet:** Du missar säkerhetsrisker
- **Kreditrisk:** Du nekar lån till många som skulle betalat tillbaka

Rättsväsendet - en känslig balans: I rättsväsendet är både precision och recall kritiska:

- **Hög precision:** Vi vill inte döma oskyldiga (false positive)
- **Hög recall:** Vi vill inte låta skyldiga gå fria (false negative)

Praktisk lösning:

- Använd F1-score som balanserar båda
- Justera tröskelvärde baserat på samhällskostnad
- Implementera flera steg i processen

8. Förklara hur man kan tolka figur 4.8 på sidan 175.

Figur 4.8 visar beslutsgränser för en logistisk regressionsmodell:

Vad grafen visar:

- **X-axel:** Variabel X1 (ranges från -2.5 till 3.5)
- **Y-axel:** Variabel X2 (ranges från -2.5 till 2.5)
- **Data-punkter:** Två klasser (klass 0 = lila/svarta cirklar, klass 1 = gula cirklar)

Bakgrundsfärgerna (sannolikhetskarta):

- **Mörk lila/blå:** Låg sannolikhet för klass 1 (nära 0.0)
- **Ljus gul:** Hög sannolikhet för klass 1 (nära 1.0)
- **Grön/blågrön övergång:** Mellanliggande sannolikheter (0.2-0.8)

Beslutsgränsen:

- **Tröskelvärde 0.5:** Gränsen mellan klass 0 och klass 1
- **Diagonal linje:** Ungefär där sannolikheten är 0.5
- **Punkter på vänster sida:** Predikteras som klass 0
- **Punkter på höger sida:** Predikteras som klass 1

Tolkning:

- **Klass 0 (lila):** Koncentrerade i övre vänstra området (låga X1, höga X2)
- **Klass 1 (gul):** Koncentrerade i nedre högra området (höga X1, låga X2)
- **Överlappning:** Mitt i grafen finns det en övergångszon med osäker klassificering

Vad detta betyder:

- Modellen skapar en **probabilistisk separation** mellan klasserna
- Beslutsgränsen är **mjuk** (gradvis övergång) inte skarp
- Modellen är **osäker** i överlappningszonen
- **X1 och X2** tillsammans avgör klassificeringen

Praktisk betydelse:

- Visar hur modellen "ser" data
- Hjälper förstå varför vissa punkter klassificeras som de gör
- Visar modellens osäkerhet i gränsområdena

9. På sidan 209 står det "på träningsdatan använder vi `.fit_transform()` , på valideringsdatan och testdatan använder vi endast `.transform()` ." Förklara logiken bakom detta.

Grundprinciper:

- `.fit()` : Lär sig parametrar från data (t.ex. medelvärde, standardavvikelse)
- `.transform()` : Applicerar de inlärd parametrarna på ny data
- `.fit_transform()` : Kombinerar båda stegen

Varför denna separation:

1. Undvika Data Leakage:

- Om vi använder `.fit_transform()` på valideringsdata lär modellen information från framtida data
- Detta ger optimistiska och missvisande resultat
- Vi simulerar inte verklig användning där vi bara har träningsdata

2. Konsistent Transformation:

- Samma parametrar (t.ex. samma medelvärde för skalning) används för alla datasets
- Validerings- och testdata transformeras på samma sätt som träningsdata
- Detta säkerställer att alla data behandlas likadant

3. Simulera Produktionsmiljö:

- I produktion har vi bara träningsdata att lära från
- Nya data måste transformeras med samma parametrar
- Vi kan inte "lära" från nya data

Exempel med StandardScaler:

```
# Träning
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Lär sig mean, std och transformera

# Validering/Test
X_val_scaled = scaler.transform(X_val) # Använd samma mean, std
X_test_scaled = scaler.transform(X_test) # Använd samma mean, std
```

Konsekvenser av fel:

- **Optimistisk prestanda:** Modellen verkar bättre än den är
- **Dålig generalisering:** Modellen fungerar sämre på riktig data
- **Missvisande utvärdering:** Felaktiga beslut baserat på felaktiga resultat

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 5: Dimensionsreduktion

Faktafrågor

1. Vad menas med curse of dimensionality?

Curse of dimensionality refererar till de problem som uppstår när man arbetar med data i höga dimensioner:

Huvudproblem:

1. **Avståndsmått blir mindre meningsfulla:** I höga dimensioner blir alla punkter ungefär lika långt från varandra
2. **Ökad beräkningskomplexitet:** Algoritmer blir exponentiellt långsammare
3. **Överanpassning:** Modeller blir mer komplexa och riskerar att memorera träningsdata
4. **Sparsitet:** Data blir glesare i höga dimensioner, vilket gör det svårare att hitta mönster
5. **Visualisering:** Omöjligt att visualisera data i mer än 3 dimensioner

Lösningar:

- Dimensionsreduktion (PCA, t-SNE)
- Feature selection
- Regularisering
- Ensemble-metoder

2. Vad är dimensionsreducering och varför görs det?

Dimensionsreducering är processen att reducera antalet features (variabler) i en dataset samtidigt som man behåller så mycket viktig information som möjligt.

Varför görs det:

1. **Reducera beräkningskomplexitet:** Färre dimensioner = snabbare beräkningar
2. **Förbättra visualisering:** Möjliggör visualisering av högdimensionell data
3. **Minska överanpassning:** Färre parametrar = mindre risk för overfitting
4. **Ta bort brus och redundans:** Eliminera irrelevanta eller korrelerade features
5. **Förbättra modellprestanda:** Bättre generalisering på nya data
6. **Hantera curse of dimensionality:** Minska problem med höga dimensioner

Metoder:

- **Feature selection:** Väljer ut viktiga features
- **Feature extraction:** Skapar nya features från befintliga (PCA, t-SNE, LDA)

3. Förklara översiktligt hur PCA fungerar. Använd figur 5.4 på sidan 224 i din förklaring.

PCA (Principal Component Analysis) är en linjär dimensionsreduktionsteknik som hittar de riktningar (huvudkomponenter) där data varierar mest.

Algoritm:

1. **Standardisering:** Centra och skala data
2. **Kovariansmatris:** Beräkna $C = (1/n) X^T X$
3. **Eigenvalues och Eigenvectors:** Hitta egenvärden och egenvektorer
4. **Projektion:** Projicera data på huvudkomponenter

Figur 5.4 visar:

- Originaldata i 2D
- Huvudkomponenter (PC1 och PC2) som riktningar med störst varians
- Projektion av data på huvudkomponenterna
- PC1 förklarar mer varians än PC2

Matematisk grund:

$$C = P \Lambda P^T$$

där P är matris med egenvektorer, Λ är diagonal matris med egenvärden.

Fördelar:

- Linjär transformation
- Bevarar global struktur
- Matematiskt välgrundad
- Snabb beräkning

Nackdelar:

- Antagande om linjära relationer
- Känslig för outliers
- Kan förlora viktig information

4. Hur kan kernel PCA utvärderas?

Kernel PCA är en utvidgning av PCA som kan hantera icke-linjära relationer genom att använda kernel-trick.

Utvärderingsmetoder:

1. Rekonstruktionsfel

- Mät hur väl data kan rekonstrueras
- Mean Squared Error (MSE)
- Jämför med originaldata

2. Variance Explained

- Andel av total varians som bevaras
- Cumulative variance plot

- Välj antal komponenter baserat på önskad variansförklaring

3. Cross-validation

- Dela data i träning/validering
- Träna på träningsdata
- Utvärdera på valideringsdata
- Undvik överanpassning

4. Downstream Task Performance

- Använd reducerade features för klassificering/regression
- Jämför prestanda med originalfeatures
- Mät accuracy, precision, recall

5. Visualisering

- 2D/3D scatter plots
- Kvalitativ bedömning av kluster
- Separation mellan klasser

6. Kernel Selection

- Testa olika kernels (RBF, polynomial, sigmoid)
- Jämför prestanda
- Välj kernel baserat på dataegenskaper

Praktiska tips:

- Standardisera data före kernel PCA
- Välj kernel-parametrar noggrant
- Överväg beräkningskostnad
- Validera resultat på oberoende data

Resonemangsfrågor

5. Stina påstår att man i maskininlärning alltid vill ha modeller som genomför så bra prediktioner som möjligt. Kalle påstår att det inte riktigt stämmer eftersom tid också är en viktig aspekt. Både för själva modellträningen och för själva prediktionerna. Vad säger du?

Kalle har rätt - tid är en kritisk aspekt i maskininlärning som ofta överskuggas av fokus på prediktionsprestanda. Här är varför:

Varför tid är viktigt:

1. Modellträning

- **Stora dataset:** Träning kan ta timmar eller dagar

- **Iterativ utveckling:** Snabbare träning = fler experiment
- **Resurskostnad:** Beräkningskraft kostar pengar
- **Deadlines:** Projekt har tidsramar

2. Prediktioner (Inference)

- **Real-time applikationer:** Måste svara inom millisekunder
- **Skalbarhet:** Tusentals användare samtidigt
- **Resursbegränsningar:** Mobilappar, edge devices
- **Användarupplevelse:** Långsam respons = dålig UX

3. Praktiska överväganden

- **Maintenance:** Komplexa modeller är svårare att underhålla
- **Deployment:** Enkla modeller är lättare att implementera
- **Debugging:** Snabbare att felsöka enkla modeller
- **Updates:** Lättare att uppdatera snabba modeller

Balans mellan prestanda och tid:

När prestanda är viktigast:

- Medicinska diagnoser
- Säkerhetskritiska system
- Offline-analys
- Batch-processing

När tid är viktigast:

- Real-time rekommendationer
- Mobilappar
- Trading-system
- Interaktiva applikationer

Praktiska lösningar:

1. **Model compression:** Kvantisering, pruning
2. **Ensemble simplification:** Färre modeller
3. **Feature selection:** Färre features
4. **Hardware optimization:** GPU, TPU
5. **Caching:** Spara vanliga prediktioner

Slutsats: Optimal balans beror på användningsfall. I verkligheten måste man ofta kompromissa mellan prestanda och hastighet.

6. Efter att vi genomfört en PCA, vad händer med tolkningen av variablerna?

Efter PCA förändras tolkningen av variablerna avsevärt, vilket är en av de största utmaningarna med metoden:

Vad händer med tolkningen:

1. Förlust av originalvariabler

- **Originalvariabler:** Tydlig tolkning (ålder, inkomst, etc.)
- **Huvudkomponenter:** Abstrakta kombinationer av originalvariabler
- **Svårt att tolka:** Vad betyder "PC1" i praktiken?

2. Matematisk transformation

- $PC1 = w_1 \times \text{variabel1} + w_2 \times \text{variabel2} + \dots + w_n \times \text{variabeln}$
- **Vikter (w):** Visar bidrag från varje originalvariabel
- **Tolkning:** Kräver analys av vikterna

3. Praktiska utmaningar

- **Affärsstakeholders:** Förstår inte "PC1"
- **Beslutsfattning:** Svårt att basera beslut på abstrakta komponenter
- **Kommunikation:** Måste översätta till förståeligt språk

Sätt att hantera tolkning:

1. Analysera komponentvikter

```
# Exempel: PC1 = 0.7×ålder + 0.5×inkomst - 0.3×utbildning  
# Tolkning: PC1 representerar "mogenhet/erfarenhet"
```

2. Skapa meningsfulla namn

- **PC1:** "Socioekonomisk status"
- **PC2:** "Livsstil"
- **PC3:** "Riskbenägenhet"

3. Visualisera bidrag

- **Heatmaps:** Visa vikter för varje komponent
- **Bar charts:** Jämför vikter mellan komponenter
- **Biplots:** Kombinera data och variabler

4. Domänkunskap

- **Expertis:** Använd domänkunskap för tolkning
- **Validering:** Kontrollera att tolkningar är rimliga
- **Iteration:** Justera baserat på feedback

Alternativa metoder för bättre tolkning:

1. Feature Selection

- Behåll originalvariabler
- Bättre tolkning

- Kan förlora information

2. LDA (Linear Discriminant Analysis)

- Övervakad metod
- Bättre tolkning
- Kräver klasslabels

3. Factor Analysis

- Modellerar latent faktorer
- Bättre tolkning
- Mer komplex

Praktiska rekommendationer:

1. **Dokumentera tolkningar:** Skriv ner vad varje komponent betyder
2. **Validera med experter:** Kontrollera att tolkningar är korrekta
3. **Använd visualisering:** Hjälp stakeholders att förstå
4. **Överväg alternativ:** Feature selection om tolkning är kritisk
5. **Kommunikera tydligt:** Förklara begränsningar och fördelar

Slutsats: PCA förbättrar prestanda men försämrar tolkning. Detta är en avvägning som måste göras baserat på projektets mål och stakeholders behov.

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 6: Klustering

Faktafrågor

1. Vad är klustering för något? Ge några exempel på tillämpningsområden.

Klustering är en oövervakad maskininlärningsteknik som grupperar liknande datapunkter tillsammans utan att veta de korrekta etiketterna i förväg. Målet är att hitta naturliga grupperingar i data.

Exempel på tillämpningsområden:

- **Kundsegmentering:** Gruppera kunder baserat på köpbeteende för anpassad marknadsföring
- **Bildkomprimering:** Reducera färgpalett med k-means för att behålla viktiga färger
- **Genanalys:** Gruppera gener med liknande uttryck för att identifiera genfunktioner
- **Dokumentklustering:** Organisera stora textsamlingar genom att gruppera liknande dokument
- **Medicinsk bildanalys:** Identifiera olika typer av celler eller vävnader
- **Säkerhetsanalys:** Detektera ovanliga mönster i nätverkstrafik
- **Rekommendationssystem:** Gruppera användare med liknande preferenser

2. Förklara översiktligt hur K-means fungerar. Använd figur 6.3 (sidan 238) och figur 6.4 (sidan 239) i din förklaring.

K-means är en iterativ klusteralgorithm som fungerar enligt följande steg:

Algorithm:

1. **Initialisering:** Välj k centroider slumpmässigt (figur 6.3 visar initiala centroider)
2. **Tilldelning:** Tilldela varje datapunkt till närmaste centroid
3. **Uppdatering:** Beräkna nya centroider som medelvärde av tilldelade punkter
4. **Iteration:** Upprepa steg 2-3 tills konvergens (figur 6.4 visar slutresultat)

Matematisk formulering:

$$J = \sum_{i=1}^k \sum_{x \in C_i} ||x - \mu_i||^2$$

där J är Within-cluster sum of squares (WCSS), k är antal kluster, C_i är kluster i, och μ_i är centroid för kluster i.

Figurerna visar:

- Figur 6.3: Initiala centroider placerade slumpmässigt
- Figur 6.4: Slutliga centroider efter konvergens med optimerade klustertilldelningar

Fördelar:

- Enkel och snabb algorithm
- Skalar bra med stora dataset

- Fungerar bra med sfäriska kluster

Nackdelar:

- Kräver att antal kluster (k) specificeras i förväg
- Känslig för initialisering
- Fungerar bara med sfäriska kluster
- Känslig för outliers

Resonemangsfrågor

3. Hur kan man välja vilket antal kluster som ska användas för en K-means modell?

Använd inertia, silhouette score och silhouette diagram i ditt svar.

Val av antal kluster är en kritisk aspekt av K-means klustring. Här är de huvudsakliga metoderna:

1. Inertia (Elbow-metoden)

- **Inertia** = Within-cluster sum of squares (WCSS)
- Plotta inertia mot antal kluster
- Välj punkt där kurvan "böjer" (elbow point)
- **Fördelar:** Enkel att implementera
- **Nackdelar:** Subjektiv, inte alltid tydlig "elbow"

2. Silhouette Score

- Mäter kvalitet på klustertilldelning
- Värden mellan -1 och 1
- Högre värden = bättre klustring
- **Beräkning för varje punkt:**
- $a(i)$ = genomsnittligt avstånd till andra punkter i samma kluster
- $b(i)$ = genomsnittligt avstånd till närmaste kluster
- $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$
- **Fördelar:** Objektiv mått, tar hänsyn till både kompakthet och separation
- **Nackdelar:** Beräkningskrävande

3. Silhouette Diagram

- Visar silhouette score för varje datapunkt
- Längd på staplar = silhouette score
- Färg = klustertillhörighet
- **Tolkning:**
- Långa staplar = bra klustertilldelning
- Korta staplar = osäker klustertilldelning
- Negativa värden = fel tilldelning

Praktisk rekommendation:

1. Börja med Elbow-metoden för grov uppskattning

2. Använd silhouette score för finjustering
3. Visualisera med silhouette diagram för detaljerad analys
4. Överväg domänkunskap och affärsmål

4. Om du kollar på figur 6.10 på sidan 247, hur många kluster hade du valt och varför? Är det en "exakt vetenskap" att välja antalet kluster?

Baserat på figur 6.10 (som visar silhouette score mot antal kluster) skulle jag välja **k=3** eftersom:

Anledningar till k=3:

1. **Högst silhouette score:** K=3 ger det högsta silhouette score-värdet
2. **Tydlig topp:** Det finns en tydlig topp vid k=3, vilket indikerar optimal klusterkvalitet
3. **Balans:** K=3 ger en bra balans mellan kompakthet och separation

Är det en "exakt vetenskap"?

Nej, det är inte en exakt vetenskap av följande anledningar:

1. **Subjektivitet:** Olika mått kan ge olika resultat
2. **Domänkunskap:** Affärsmål och domänkunskap påverkar valet
3. **Dataegenskaper:** Olika dataset kräver olika antal kluster
4. **Kompromisser:** Måste balansera modellkomplexitet mot tolkbarhet
5. **Iterativ process:** Ofta behöver man testa flera alternativ

Praktiska överväganden:

- **Tolkbarhet:** Färre kluster är ofta lättare att tolka
- **Affärsmål:** Antal kluster ska matcha affärsbehov
- **Datakvalitet:** Bättre data = mer tillförlitliga resultat
- **Validering:** Testa resultatet på nya data

5. Hur tolkar man figur 6.13 på sidan 251?

Figur 6.13 visar ett **silhouette diagram** som hjälper till att bedöma klusterkvalitet. Här är hur man tolkar det:

Struktur av diagrammet:

- **Y-axeln:** Datapunkter (sorterade per kluster)
- **X-axeln:** Silhouette score (-1 till 1)
- **Färger:** Olika kluster
- **Staplar:** Silhouette score för varje datapunkt

Tolkning av resultat:

1. Längd på staplar:

- **Långa staplar:** Bra klustertilldelning
- **Korta staplar:** Osäker klustertilldelning
- **Negativa staplar:** Fel tilldelning (punkt borde vara i annat kluster)

2. Färgkonsistens:

- **Enhetlig färg per kluster:** Bra separation
- **Blandade färger:** Osäker klustertilldelning

3. Klusterstorlek:

- **Jämna storlekar:** Balanserade kluster
- **Ojämna storlekar:** Kan indikera problem

4. Genomsnittlig silhouette score:

- **Högt värde (>0.5):** Bra klustring
- **Medelvärde ($0.2-0.5$):** Acceptabel klustring
- **Lågt värde (<0.2):** Dålig klustring

Praktiska insikter:

- **Kluster med många negativa värden:** Överväg att minska antal kluster
- **Kluster med varierande längder:** Kan indikera att klustret inte är homogent
- **Jämna staplar:** Indikerar stabil klustring

Förbättringsåtgärder:

1. **Justera antal kluster:** Testa fler/färre kluster
 2. **Förbättra data:** Standardisera eller rensa data
 3. **Välj annan algoritm:** Testa hierarkisk klustring eller DBSCAN
 4. **Feature engineering:** Skapa bättre features
-

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 7: Artificiella Neurala Nätverk (ANN)

FAKTAFRÅGOR

1. Vad har ANN modellerna inspirerats av?

ANN-modellerna (Artificiella Neurala Nätverk) har inspirerats av hur den mänskliga hjärnan fungerar med dess neuronnät. I presentationstalet för Nobelpriset i fysik 2024 sägs det att årets Nobelpristagare, John Hopfield och Geoffrey Hinton, inspirerades av neuronnätverket i den mänskliga hjärnan när de utvecklade artificiella neurala nätverk.

Liknelsen mellan biologiska neuroner och artificiella neurala nätverk:

- **Input layer:** Motsvarar sinnesintryck - vi får in något (t.ex. se en farlig tiger)
- **Hidden layer:** Bearbetning av det som kommit in - "dolda" tankeprocesser (t.ex. tänka att tigern är farlig)
- **Output layer:** Slutsats efter bearbetning - vi får ut något (t.ex. beslutet att springa)

Det är viktigt att notera att liknelsen med hjärnan är användbar för intuition, men ANN fungerar faktiskt inte på samma sätt som hjärnan - det är en inspiration, inte en exakt kopia.

2. Vad refererar "djup" till i begreppet "djupinlärning"?

"Djup" i begreppet "djupinlärning" (Deep Learning, DL) refererar till det faktum att neurala nätverk kan ha många dolda lager (hidden layers). Det finns ingen exakt definition på hur många lager som krävs för att ett neuralt nätverk ska klassas som "djupt", men i praktiken anses ett neuralt nätverk med flera dolda lager vara "djupt".

Viktiga punkter: - Djupinlärning = neurala nätverk med många dolda lager - Begreppen "djupinlärning" och "neurala nätverk" används ofta synonymt - Ett single-layer perceptron har inga dolda lager - Ett Multilayer Perceptron (MLP) har flera dolda lager och kan därför klassas som djupinlärning

3. Förklara vad som händer i figur 7.2, 7.3, figur 7.4 och figur 7.5. Varför används aktiveringsfunktioner?

Figur 7.2: Enkel linjär regression

- Visar ett neuralt nätverk utan aktiveringsfunktioner
- Input-noderna multipliceras med vikter (W_1 , W_2) och adderar en bias
- Outputen blir: $y = W_1 + W_2 \cdot x_1$
- **Problemet:** Detta är exakt samma som linjär regression - vi har inte fått något nytt!

Figur 7.3: Mer komplex arkitektur (fortfarande utan aktiveringsfunktioner)

- Har ett dolt lager med flera noder
- Trots den ökade komplexiteten blir outputen fortfarande: $y = \theta_1 + \theta_2 \cdot x_1$
- **Problemet:** Även med dolda lager får vi samma resultat som linjär regression om vi inte använder aktiveringsfunktioner

Figur 7.4: Aktiveringsfunktioner

- Visar två viktiga aktiveringsfunktioner:
- **Logistic/Sigmoid:** $\sigma(z) = 1 / (1 + e^{-z})$ - ger värden mellan 0 och 1
- **ReLU (Rectified Linear Unit):** $\text{ReLU}(z) = \max(0, z)$ - ger värden ≥ 0
- Visar hur dessa funktioner ser ut grafiskt

Figur 7.5: Single-layer perceptron med aktiveringsfunktioner

- Visar en arkitektur som faktiskt skiljer sig från linjär regression
- Karakteriseras av:
- Feedforward (data går framåt från input till output)
- Fully connected/dense (alla noder är kopplade)
- **Ikke-linjära aktiveringsfunktioner** (detta är nyckeln!)

Varför används aktiveringsfunktioner?

Aktiveringsfunktioner introducerar **ikke-linearitet** i neurala nätverk. Utan aktiveringsfunktioner skulle ett neuralt nätverk bara kunna modellera linjära samband - precis som linjär regression. Med ikke-linjära aktiveringsfunktioner kan nätverket lära sig komplexa, ikke-linjära samband mellan input och output.

Två viktiga aktiveringsfunktioner: 1. **Logistic/Sigmoid:** Används för binär klassificering (kan tolkas som sannolikhet) 2. **ReLU:** Standardval för hidden layers i de flesta fall

4. Har neurala nätverk få eller många parametrar?

Neurala nätverk har **väldigt många parametrar**. Detta är en av de mest karakteristiska egenskaperna hos neurala nätverk.

Exempel: I ett exempel med MLPRegressor med: - 20 input features - Hidden layers: (200, 100, 10) - 1 output

Får vi totalt **25,321 parametrar**: - 25,010 vikter (weights) - 311 bias-värden

Jämförelse: - En linjär regressionsmodell med 20 features har bara 21 parametrar (20 vikter + 1 bias) - Neurala nätverk har ofta tusentals eller miljoner parametrar

Konsekvenser av många parametrar: 1. **Mer data behövs:** Ju fler parametrar, desto mer data behövs för att träna modellen bra 2. **Ökad risk för överanpassning:** Många parametrar ökar risken att modellen blir överanpassad (overfitted) eftersom modellen blir för komplex i förhållande till datans faktiska komplexitet 3.

Behov av regularisering: Därför behövs tekniker som dropout, L1/L2 regularisering, early stopping och batch normalization

5. Förklara intuitivt hur dropout-regularisering fungerar.

Dropout är en populär metod för att regularisera neurala nätverk och minska överanpassning.

Hur det fungerar: - Vid varje träningsiteration har varje neuron en sannolikhet p (vanligtvis 10-50%) att bli "droppad" - En droppad neuron har output 0 och deltar inte i beräkningarna under den iterationen - Detta sker slumpmässigt för varje iteration

Intuition: Detta tvingar neuronerna att "lära sig själva" och inte förlita sig för mycket på andra neuroner. Det förhindrar så kallad **co-adaptation** där neuroner blir beroende av varandra på ett sätt som kan leda till överanpassning.

Praktiskt: - Om modellen är överanpassad: höj dropout-graden (t.ex. från 0.3 till 0.5) - Vanliga värden: 10-50% (0.1 till 0.5) - Dropout används endast under träning, inte vid prediktion

Exempel: Om vi har dropout rate = 0.3 (30%), så kommer vid varje träningsiteration 30% av neuronerna slumpmässigt att stängas av. Detta gör att nätverket måste lära sig att fungera även när vissa neuroner saknas, vilket gör modellen mer robust.

RESONEMANGSFRÅGOR

6. Din kollega ber dig förklara tabell 7.1 och tabell 7.2. Gör det!

Tabell 7.1: Typisk MLP-arkitektur för regressionsproblem

Antal input neuroner: - En per input feature - Exempel: För MNIST (28x28 pixlar) = 784 input neuroner

Antal hidden layers: - Beror på problemet, men vanligtvis 1 till 5 lager

Antal neuroner per hidden layer: - Beror på problemet, men vanligtvis 10 till 100

Antal output neuroner: - 1 (eller flera om flera dimensioner ska predikteras)

Hidden activation: - ReLU (eller SELU) - standardval för dolda lager

Output activation: - Ingen aktiveringsfunktion, eller - ReLU (när y är positivt - t.ex. ålder, lön), eller - Logistisk/tanh (när y är i ett begränsat intervall)

Loss-funktion: - MSE (Mean Squared Error) eller - MAE (Mean Absolute Error) om det finns outliers

Exempel på användning: Om vi vill prediktera någons ålder (som alltid är ≥ 0), kan vi använda ReLU som output activation eftersom ReLU alltid ger värden ≥ 0 .

Tabell 7.2: Typisk MLP-arkitektur för klassificeringsproblem

Tabellen visar tre typer av klassificeringsproblem:

A. Binär klassificering: - **Output neuroner:** 1 - **Output activation:** Logistic (sigmoid) - **Loss-funktion:** Cross entropy - **Exempel:** Har personen en hund? (Ja/Nej)

B. Multilabel binär klassificering: - **Output neuroner:** 1 per label - **Output activation:** Logistic (sigmoid) för varje output - **Loss-funktion:** Cross entropy - **Exempel:** Vilka djur har personen? (Kan ha både hund OCH katt)

C. Multiklass klassificering: - **Output neuroner:** 1 per klass - **Output activation:** Softmax (generalisering av sigmoid där summan av alla outputs blir 1) - **Loss-funktion:** Cross entropy - **Exempel:** Vilken siffra är det? (0, 1, 2, ..., 9 - endast EN siffra)

Gemensamt för alla klassificeringsproblem: - Antal input neuroner: Samma som för regression (en per feature) - Antal hidden layers och neuroner: Samma som för regression (1-5 lager, 10-100 neuroner)

Viktig skillnad: - **Softmax** används för multiklass eftersom den ger en sannolikhetsfördelning där summan av alla outputs = 1.0 - **Logistic (sigmoid)** används för binär klassificering eftersom den ger ett värde mellan 0 och 1 som kan tolkas som sannolikhet

7. Experimentera med neurala nätverk på följande länk:

<https://playground.tensorflow.org/>

Instruktioner: Denna uppgift kräver att du själv går in på TensorFlow Playground och experimenterar. Här är några saker du kan prova:

Experiment att göra: 1. **Testa olika aktiveringsfunktioner:** Jämför ReLU, Tanh, Sigmoid - se hur de påverkar modellens förmåga att lära sig 2. **Variera antal hidden layers:** Testa 1, 2, 3, 4 lager - se hur djupare nätverk kan lära sig mer komplexa mönster 3. **Variera antal neuroner per lager:** Se hur fler neuroner kan hjälpa modellen 4. **Testa olika datasets:** Prova de olika datamönstrena (spiral, cirkel, XOR, etc.) 5. **Lägg till noise:** Se hur modellen hanterar brusig data 6. **Observera learning rate:** Se hur learning rate påverkar träningsprocessen

Vad du ska lära dig: - Intuition för hur neurala nätverk fungerar visuellt - Hur olika hyperparametrar påverkar träningen - Hur modellen lär sig att separera olika klasser - Hur aktiveringsfunktioner introducerar icke-linearitet

Reflektion: Efter att ha experimenterat, reflektera över: - När behövs fler lager? - När behövs fler neuroner? - Hur påverkar learning rate träningen? - Vilka mönster är svårast att lära sig?

8. Förklara översiktligt hur backpropagation fungerar. Använd figur 7.7 till din hjälp.

Backpropagation är algoritmen som används för att träna neurala nätverk. Den är grunden för hur neurala nätverk lär sig.

Steg-för-steg förklaring:

1. Batch-hantering

- Algoritmen hanterar en batch av data (t.ex. 32 observationer) åt gången
- Detta gör träningen mer effektiv än att hantera en observation i taget

2. Forward pass (framåtpassering)

- Input-datan färdas framåt genom nätverket
- Varje lager beräknar sina värden baserat på föregående lager
- Man gör en prediktion baserat på nuvarande vikter

3. Loss-beräkning

- Felet mäts genom att jämföra sanna värden med predikterade värden
- Detta görs via loss-funktionen (t.ex. MSE för regression, Cross entropy för klassificering)
- Loss-funktionen ger ett mått på hur fel modellen var

4. Backward pass (bakåtpassering) - "backpropagation"

- Algoritmen går **tillbaka** genom nätverket (därav namnet "backpropagation")
- Den beräknar hur varje vikt har bidragit till det totala felet
- Detta görs genom att använda kedjeregeln (chain rule) från kalkylen
- Varje vikt får ett "ansvar" för felet - vikter som bidrog mycket till felet får större ansvar

5. Gradient Descent

- Baserat på "ansvaret" för felet, justeras vikterna så att felet minskar
- Learning rate bestämmer hur stora justeringar som görs
- Vikterna uppdateras i riktning som minskar felet

Intuition: Tänk dig att du skjuter på en måltavla: - **Forward pass:** Du skjuter och ser var pilen landade - **Loss-beräkning:** Du mäter hur långt från mittpunkten pilen landade - **Backpropagation:** Du analyserar vilka faktorer (vikter) som gjorde att pilen hamnade fel - **Gradient Descent:** Du justerar ditt siktande (vikterna) för att nästa skott ska bli bättre

Varför "backward"? Algoritmen måste gå bakåt eftersom felet uppstår i output-lagret, men vi behöver veta hur alla lager bidrog till felet. Genom att gå bakåt kan vi sprida "ansvaret" för felet tillbaka genom alla lager.

Viktiga begrepp: - **Gradient:** Riktingen och storleken på förändringen som behövs för att minska felet - **Chain rule:** Matematisk regel som gör det möjligt att beräkna gradienter i komplexa nätverk - **Learning rate:** Hur stora steg vi tar när vi uppdaterar vikterna

Praktiskt: - Backpropagation är automatiserad i Keras/TensorFlow - vi behöver inte implementera den själva -
Men förståelsen för hur den fungerar hjälper oss att förstå varför vissa saker fungerar (t.ex. varför vi behöver icke-
linjära aktiveringsfunktioner)

Sammanfattning

Faktafrågor handlar om grundläggande kunskaper om ANN: - Inspiration från hjärnan - Vad "djup" betyder - Varför aktiveringsfunktioner behövs - Att neurala nätverk har många parametrar - Hur dropout fungerar

Resonemangsfrågor kräver djupare förståelse: - Tolka och förklara riktlinjer för modellarkitektur (tabellerna) - Experimentera och lära sig praktiskt - Förstå grundläggande algoritmer (backpropagation)

Genom att besvara dessa frågor får man en solid grund för att arbeta med neurala nätverk i praktiken.

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 8: CNN

Faktafrågor

1. Vad krävs det för att ett neuralt nätverk ska klassas som CNN?

För att ett neuralt nätverk ska klassas som CNN (Convolutional Neural Network) behöver det minst ett convolution lager som ett av de dolda lagren. CNN är en klass av artificiella neurala nätverk (ANN), och det är närvaron av convolution layers som skiljer CNN från vanliga ANN-modeller.

2. Inom vilket tillämpningsområde är CNN generellt sett en väldigt kraftfull modell?

CNN är generellt sett väldigt kraftfulla inom datorseende (computer vision). CNN inspirerades av den del av hjärnan som hanterar vision eller datorseende. Som tumregel kan vi säga att om vi arbetar med visuell data (bilder eller videor) så är CNN den modell vi börjar med. CNN presterar mycket bra när vi arbetar med bilder eller videor, exempelvis för klassificering där man vill identifiera vad det är på en bild.

3. Vad menas med RGB?

RGB står för Röd, Grön och Blå. Det är tre kanaler som representerar grundfärgerna i additiv färgblandning. När vi arbetar med färgbilder i CNN så består varje bild ofta av tre kanaler som representerar dessa färger. Exempelvis kan en bild ha storleken (32 x 32) och tre kanaler (RGB), vilket ger oss en bild med dimensionerna (32, 32, 3).

4. Vad är data augmentation?

Data augmentation är en användbar teknik när vi arbetar med bild-data. Det innebär att vi förvränger bilder på olika sätt, exempelvis: - Roterar dem - Ändrar ljusstyrkan - Zoomar in/ut

Syftet med data augmentation är att få fler bilder till vår träningsdata, vilket skyddar modellen mot att bli överanpassad. Detta gör det till en form av regularisering. Viktigt är att bilderna endast förvrängas så pass lite att en människas förmåga att tolka bilden inte påverkas.

Resonemangsfrågor

5. Förklara översiktligt hur CNN fungerar. Använd begreppen convolution layer och pooling layer i ditt svar.

CNN fungerar genom att först identifiera low-level features som enklare färger och former. Dessa mer enkla egenskaper kombineras därefter för att identifiera high-level features såsom ögon, mun, öron och dylikt.

Modellen använder convolutional layers, ofta i kombination med pooling layers.

Convolution layers: Ett convolutional lager består av flera filter där varje filter "söker efter" och betonar vissa lokala attribut/egenskaper. I praktiken används flera filter för att hitta flera olika attribut, och när nätverket tränas så lär det sig vilka filter som är bra att använda. Filtret tillämpas på varje delmatris i bilden genom elementvis multiplikation och addition. Om en delmatris i originalbilden liknar filtret så kommer vi få ett högt värde; annars ett litet värde. Således fokuserar convolved image på de delar i en bild som liknar filtret som tillämpas.

Pooling layers: Pooling layers fokuserar på de viktigaste delarna i en bild. Resultatet blir att bildens storlek minskar. Max pooling layer tar maximumvärdet för varje icke-överlappande delmatris. Om vi använder ett (2 x 2) max pooling tas alltså maximumvärdet för varje icke-överlappande (2 x 2) delmatris.

Genom att kombinera convolution layers och pooling layers kan CNN gradvis bygga upp mer komplexa representationer av bilden, från enkla mönster till komplexa objekt.

6. Din kollega ber dig förklara figur 8.4. Gör det!

Figur 8.4 visar en exempel på en CNN-modellarkitektur för 100 klasser. Här är en steg-för-steg förklaring:

1. **Input-lagret:** Vi ser en bild med storleken (32 x 32) som består av tre kanaler som representerar färgerna röd, grön och blå (RGB). Generellt sett används dessa som grundfärger i additiv färgblandning.
2. **Convolution layer:** För varje kanal tillämpas ett convolution lager med två filter. Därför får vi nu sex kanaler totalt (3 kanaler × 2 filter = 6 kanaler).
3. **Max pooling:** Max pooling tillämpas där storleken (2 x 2) används för filtreringen. Den nya bildstorleken blir därför (16 x 16).
4. **Convolution layer:** Ett convolution lager med två filter tillämpas återigen vilket ger oss 12 stycken kanaler.
5. **Max pooling:** Max pooling tillämpas där storleken (2 x 2) används för filtreringen. Den nya bildstorleken blir därför (8 x 8).
6. **Convolution layer:** Ett convolution lager med två filter tillämpas återigen vilket ger oss 24 stycken kanaler.
7. **Max pooling:** Max pooling tillämpas där storleken (2 x 2) används för filtreringen. Den nya bildstorleken blir därför (4 x 4) vilket gör att varje bild består av totalt 16 pixlar.
8. **Flatten layer:** Ett flatten layer tillämpas så att pixlarna från de två dimensionella bilderna tas ut till en dimension.
9. **Output-lager:** I sista steget har vi ett output-lager med 100 stycken noder eftersom vi antog att datan vi arbetar med består av 100 klasser. Vi använder softmax som aktiveringsfunktion när vi arbetar med multiklass klassificering.

Notera att vi använder fler filter för varje efterföljande Conv2D-lager (2, 2, 2 filter i exemplet, men i praktiken ofta 32, 64, 128, 256). Anledningen är att efter varje MaxPooling2D-lager så sjunker dimensionen på bilderna, vilket kompenseras med fler kanaler.

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 9: RNN

Faktafrågor

1. Rent generellt, vad är RNN-modeller designade för att användas till?

RNN-modeller är designade för att hantera sekventiell data såsom: - Text - Tal - Tidsserier

I alla dessa fall är ordningen av datan viktig. Exempelvis har texten "Jag gillar att lära mig nya saker" en helt annan innebörd än "mig saker jag nya att gillar lära". RNN-modeller har ett minne som gör att sekventiell data där ordningen spelar roll kan hanteras.

2. Vad heter två populära modellarkitekturer inom RNN?

Två populära modellarkitekturer inom RNN är: - **LSTM** (Long Short-Term Memory) - **GRU** (Gated Recurrent Unit)

LSTM är generellt sett den mest använda RNN-arkitekturen. LSTM-arkitekturer har ett mer avancerat minne och kan lära sig vad som är viktigt att komma ihåg och vad som kan glömmas bort. GRU är en förenklad variant av LSTM men har i många sammanhang visat sig fungera lika bra som LSTM.

3. Vad står NLP för? Vad är embeddings?

NLP står för **Natural Language Processing** (naturlig språkbehandling på svenska). NLP handlar om att hantera mänskligt språk med hjälp av datorer. Det finns flera tillämpningsområden, exempelvis sentimentanalys där man klassificerar om en film anses vara bra eller dålig beroende på dess omdöme.

Embeddings (ord-embeddings eller word embeddings) är numeriska representationer av ord. För att kunna använda text-data i våra modeller måste vi först transformera texten till siffror. Istället för one-hot-encoding (som ger en hög-dimensionell matris som består av mestadels nollor) använder vi word-embedding. Det innebär att ord representeras som numeriska vektorer där vektorerna ska reflektera ordens semantiska betydelse. Ord som ligger nära varandra i betydelse ska också ha embeddings som ligger närmare varandra rent matematiskt. Exempelvis är "Varg" och "Hund" lika eftersom båda är djur, medan "Tiger" och "Katt" också är lika eftersom båda är vilddjur. Avståndet mellan dessa ord i embedding-rummet reflekterar deras semantiska likhet.

I praktiken kan vi använda ett embedding layer där vikterna tränas från datan, eller använda förtränade embeddings såsom Word2vec eller GloVe.

Resonemangsfrågor

4. Din kollega ber dig förklara figur 9.2. Gör det!

Figur 9.2 visar en visualisering av hur en RNN-modell ser ut genom tid (på engelska brukar detta beskrivas som "unfolded in time").

I figuren ser vi hur outputen från tidigare tidssteg används som input till nästa tidssteg. Specifikt: - Outputen $y(t-3)$ är resultatet av inputen $x(t-3)$ - Outputen $y(t-2)$ är resultatet av inputen $x(t-2)$ och outputen $y(t-3)$ - Outputen $y(t-1)$ är resultatet av inputen $x(t-1)$ och outputen $y(t-2)$ - Outputen $y(t)$ är resultatet av inputen $x(t)$ och outputen $y(t-1)$

Detta visar att outputen för ett tidssteg beror på outputen i föregående tidssteg (som i sin tur beror på outputen i föregående tidssteg). Detta skapar ett minne i RNN-modellen, vilket möjliggör att sekventiell data där ordningen spelar roll kan hanteras.

Detta är skillnaden mellan RNN och feedforward neurala nätverk (ANN och CNN). I feedforward-nätverk flödar datan bara framåt från input till output, medan RNN har återkoppling där output från tidigare tidssteg används som input till nästa tidssteg.

Svar på Faktafrågor och Resonemangsfrågor -

Kapitel 10: Chattbottar

Faktafrågor

1. Vad är prompt engineering?

Prompt engineering innebär att vara så specifik, deskriptiv och detaljerad som möjligt om önskad kontext, utfall, längd, format, stil med mera när man ställer frågor till chattbottar. Ett gammalt talesätt säger "som man frågar får man svar", och detta gäller även chattbottar. Generellt sett kan vi ställa de frågor vi har och få rimliga svar utan att behöva tänka alltför mycket, men om vi vill ha bättre svar kan vi försöka ställa frågor på ett mer strukturerat sätt.

Exempel: - Mindre effektiv: "skriv en dikt om AI" - Mer effektiv: "Skriv en kort och inspirerande dikt om AI som fokuserar på dess möjligheter och risker inom utbildning. Stilen ska likna Shakespeares."

2. Vad är RAG?

RAG står för **Retrieval Augmented Generation** (hämtningsförstärkt generering). RAG är en teknik som låter oss anpassa chattbottens svar utifrån en given kontext, exempelvis utifrån egna dokument.

RAG-tekniken fungerar så att om vi ger modellen en kontext (ett eller flera dokument) och säger åt modellen att enbart svara utifrån den givna kontexten. Om modellen inte hittar informationen i kontexten ska den säga "Det vet jag inte" istället för att gissa.

En RAG-modell innehåller två delar: 1. **Retriever**: Söker efter relevanta stycken i en större text 2. **Generator**: Genererar svaren utifrån den givna kontexten

RAG-tekniken använder chunking (dela upp text i mindre delar), embeddings (numeriska representationer av texten), och semantisk sökning (ofta med cosinuslikhet) för att hitta de mest relevanta delarna av texten att använda som kontext.

3. Vad är chunking och embeddings?

Chunking är processen att dela upp en lång text i mindre delar, som brukar kallas chunks. Detta behövs eftersom vi småningom kommer att söka efter de delar av texten som innehåller den information vi är intresserade av, så kallad semantisk sökning. Det finns flera strategier för chunking:

1. **Fixed-length chunking**: Delar upp texten i ett antal lika stora delar baserat på tokens, ord eller tecken. Vanligtvis låter man delarna överlappa med ett antal tecken.
2. **Sentence-based chunking**: Delar upp texten i meningar med Pythons `split()`-funktion.
3. **Semantic chunking**: Börjar med sentence-based chunking, skapar embeddings av meningarna och jämför deras innehåll med en semantisk sökning. Utifrån resultaten skapas chunks som innehåller meningar som ligger nära varandra i betydelse.

Embeddings är numeriska representationer av texten. I RAG-sammanhang behöver vi skapa embeddings (numeriska representationer av texten) innan vi kan använda dem som kontext till vår RAG-modell. Embeddings fångar textens betydelse: chunks som ligger närmare varandra i betydelse också har embeddings som ligger närmare varandra rent matematiskt. Detta gör det möjligt att göra semantiska sökningar där vi hittar textstycken som är semantiskt lika vår fråga, även om de inte innehåller exakt samma ord.

Resonemangsfrågor

4. Förklara översiktligt hur man kan evaluera en chattbot.

För att utvärdera om en chattbot (eller specifikt en RAG-modell) fungerar som vi vill kan vi använda följande metod:

1. **Skapa testdata:** Skriv ett antal frågor samt svar på frågorna (ideal answers eller ground truth).
2. **Låta modellen svara:** Låt modellen svara på frågorna.
3. **Utvärdera svaren:** Använd en annan modell (eller manuell bedömning) för att utvärdera om svaren är i linje med det vi önskar.
4. **Sätt betyg:** Sätt "betyg" på svaren, t.ex.:
 5. 1 om väldigt nära det önskade svaret
 6. 0.5 om delvis i linje
 7. 0 om felaktigt

Detta ger oss data att jämföra olika modeller med och hjälper oss att förbättra chattboten systematiskt.

Andra aspekter att utvärdera kan vara: - **Relevans:** Är svaren relevanta för frågan? - **Korrekthet:** Är informationen korrekt? - **Fullständighet:** Svarar modellen på hela frågan? - **Klarhet:** Är svaren tydliga och begripliga? - **Håller sig till kontexten:** För RAG-modeller, håller sig svaren till den givna kontexten?

5. Din kollega frågar dig vad ELIZA och Turingtestet är. Läs nedanstående två länkar innan du besvarar frågan.

ELIZA var en av de tidigaste chattbottarna, utvecklad av Joseph Weizenbaum vid MIT på 1960-talet. ELIZA simulerade en psykoanalytiker och använde enkla regler för att omformulera användarens påståenden som frågor. Trots sin enkelhet kunde många människor bli övertygade om att de pratade med en verklig person. ELIZA visade tidigt på potentialen och begränsningarna hos konversationsbaserad AI.

Turingtestet är ett test som föreslogs av Alan Turing 1950 för att avgöra om en maskin kan tänkas. Testet går ut på att en människa (bedömaren) har konversationer med både en människa och en maskin via text, utan att veta vilken som är vilken. Om bedömaren inte kan skilja mellan människan och maskinen på ett tillförlitligt sätt, anses maskinen ha klarat testet. Turingtestet har varit en viktig referenspunkt i diskussionen om artificiell intelligens och vad det innebär att vara intelligent. Moderna chattbottar som ChatGPT har visat sig kunna lura många människor i liknande tester, vilket har återupplivat diskussionen om vad intelligens verkligen innebär.

6. Ge konkreta exempel på hur olika företag och organisationer kan ta nytta av chattbottar samt vilka risker som finns.

Exempel på användningsområden:

1. **Kundservice:** Företag kan använda chattbottar för att svara på vanliga frågor, hantera bokningar, och ge support dygnet runt. Detta kan minska kostnader och förbättra kundnöjdheten.
2. **Intern kunskapsbas:** Organisationer kan använda RAG-baserade chattbottar för att låta anställda söka i interna dokument, policyer och procedurer. Exempelvis kan en chattbot hjälpa anställda att hitta information om företagets HR-policyer.
3. **Utbildning:** Skolor och universitet kan använda chattbottar som läxhjälp, för att förklara koncept, eller för att ge feedback på studenters arbete.
4. **Hälsovård:** Chattbottar kan användas för att ge grundläggande hälsoråd, påminna patienter om medicinering, eller hjälpa till med att boka tider.
5. **E-handel:** E-handelsföretag kan använda chattbottar för att hjälpa kunder att hitta produkter, ge produktrekommendationer, och hantera returer.

Risker:

1. **Felaktig information:** Chattbottar kan ge felaktiga eller missvisande svar, särskilt om de inte håller sig till sin kontext (för RAG-modeller) eller om träningsdatan innehåller felaktigheter.
2. **Säkerhetsrisker:** Känslig information kan exponeras om chattbottar inte är korrekt konfigurerade, särskilt vid användning av molnbaserade modeller.
3. **Bias och diskriminering:** Chattbottar kan återspegla bias från träningsdatan, vilket kan leda till diskriminerande eller opassande svar.
4. **Beroende och dehumanisering:** Överdriven användning av chattbottar kan minska mänsklig interaktion och skapa beroende.
5. **Arbetsförlust:** Automatisering av kundservice och andra uppgifter kan leda till att människor förlorar sina jobb.
6. **Manipulation och desinformation:** Chattbottar kan användas för att sprida desinformation eller manipulera människor.
7. **Etiska problem:** Användning av chattbottar i känsliga sammanhang (som hälsovård eller juridik) kan leda till etiska problem om de inte används korrekt eller om ansvaret inte är tydligt definierat.
8. **Tekniska begränsningar:** Chattbottar förstår inte alltid kontexten korrekt och kan ge svar som verkar rimliga men är felaktiga eller opassande.

För att minska dessa risker är det viktigt att: - Använda RAG för att begränsa svaren till specifik kontext - Implementera korrekt säkerhet och dataskydd - Ha mänsklig översyn och kvalitetskontroll - Utbilda användare om chattbottens begränsningar - Följa etiska riktlinjer och regelverk